



Trabalho Prático

Computação Gráfica - Fase II

2025/2026

Grupo 50

Francisco Contente - A106846 Gustavo Braga - A107379 Lucas Pinto - A107288

Índice Geral

1. Introdução	1
2. Engine	2
2.1. Estruturas de Dados Hierárquicas	2
2.2. Parser XML Recursivo	3
2.3. Renderização da Cena	3
3. Generator	4
3.1. Torus	4
3.1.1. Discretização da Superfície	4
3.1.2. Definição dos Ângulos	4
3.1.3. Triangulação da Malha	5
4. Sistema Solar	6
5. Resultado dos Testes	8
6. Conclusão	9

Índice de Figuras

Figura 1 - Representação de um quadrilátero [ABCD] na superfície do torus	5
Figura 2 - Cenário do Sistema Solar	6
Figura 3 - Teste 1	8
Figura 4 - Teste 2	8
Figura 5 - Teste 3	8
Figura 6 - Teste 4	8

1. Introdução

O presente relatório foi elaborado no âmbito da unidade curricular de Computação Gráfica (CG), do 3º ano, 2º semestre, do curso de Licenciatura em Engenharia Informática (LEI), no ano letivo 2025/2026. Este descreve o trabalho desenvolvido na Fase 2 do projeto desta unidade curricular.

Esta fase consistiu na evolução do motor gráfico para suportar a representação de cenas complexas através de transformações geométricas hierárquicas. Foi desenvolvido um ficheiro XML representativo de um Sistema Solar, servindo como caso prático para validar a correta herança de transformações entre planetas e satélites.

2. Engine

2.1. Estruturas de Dados Hierárquicas

A transição da Fase 1 para a Fase 2 exigiu o desenvolvimento de novas estruturas de dados internas do motor gráfico, de forma a suportar não apenas o armazenamento de geometria, mas também a representação explícita de relações de parentesco entre objetos e a sequência ordenada de transformações geométricas associadas a cada um.

```
struct Transformation {  
    std::string type;  
    float x, y, z, angle;  
};
```

Esta estrutura armazena os parâmetros de cada operação geométrica. O campo *type* permite distinguir entre translação, rotação e escala, enquanto os campos *x*, *y*, *z*, e *angle* guardam os valores numéricos correspondentes.

```
struct Group {  
    std::vector<Transformation> transforms;  
    std::vector<Model> models;  
    std::vector<Group> children;  
};
```

A estrutura *Group* constitui a unidade fundamental da hierarquia de cena. Cada nodo desta estrutura encapsula três componentes essenciais: o campo *transforms*, que armazena o conjunto ordenado de transformações; o campo *models*, que contém a lista de modelos geométricos a renderizar nesse referencial; e o campo *children*, que agrega os subgrupos descendentes, os quais herdarão, por composição, todas as transformações acumuladas ao longo do caminho desde a raiz até ao nodo pai.

```
struct Scene {  
    Window window;  
    Camera camera;  
    Group root;  
};
```

A estrutura *Scene* representa a cena completa a ser renderizada pelo motor gráfico. O campo *window* define os parâmetros da janela de visualização. O campo *camera* determina o ponto de vista e a forma como os objetos são projetados. O campo *root* é o grupo raiz da hierarquia, a partir do qual todos os objetos e transformações são organizados. Assim, *Scene* centraliza a configuração de visualização e a estrutura espacial da cena.

2.2. Parser XML Recursivo

Para suportar a leitura de ficheiros XML com hierarquias de grupos, foram realizadas duas alterações principais: a modificação da função *parseXML* e a introdução da nova função *parseGroup*.

A função *parseXML* foi reformulada para deixar de processar uma lista plana de modelos e passar a construir a raiz da hierarquia de cena. Após carregar os dados da janela e da câmara, a função itera sobre todos os grupos de primeiro nível presentes no ficheiro XML, delegando o processamento de cada um à função *parseGroup*.

Esta função constrói a árvore de objetos de forma recursiva. Para cada grupo, as transformações geométricas são processadas seguindo rigorosamente a ordem do XML, salvaguardando a sua natureza não-comutativa. Após aplicar as transformações, a função carrega os modelos 3D associados e invoca-se recursivamente para processar eventuais subgrupos. Este mecanismo de descida na árvore assegura a herança correta das transformações ao longo de toda a hierarquia, independentemente da sua profundidade.

2.3. Renderização da Cena

A renderização da cena é efetuada pela função recursiva *renderGroup*, que percorre a estrutura de dados em profundidade. O objetivo principal desta função é garantir que as transformações geométricas aplicadas a um grupo pai sejam herdadas por todos os seus descendentes.

A função segue uma sequência de três etapas bem definidas. Em primeiro lugar, invoca *glPushMatrix()*, preservando o estado atual da matriz de transformação na pilha do OpenGL. De seguida, aplica as transformações geométricas do grupo pela ordem em que foram definidas no ficheiro XML. Após transformar o referencial, procede ao desenho dos modelos geométricos do grupo atual, submetendo os vértices de cada modelo através de primitivas *GL_TRIANGLES*. Por fim, invoca-se recursivamente sobre cada subgrupo filho, propagando o referencial transformado para os descendentes, e conclui com *glPopMatrix()*, que restaura a matriz ao estado anterior, garantindo que as transformações aplicadas num grupo não se propagam para os seus grupos irmãos.

3. Generator

Antes de avançar para a representação do Sistema Solar, é necessário construir todas as primitivas geométricas que o constituem. Nesta fase, foi desenvolvida a primitiva torus, utilizada para representar o anel de Saturno.

3.1. Torus

O torus é uma superfície definida por dois parâmetros radiais: o raio maior R , que representa a distância do centro do tubo ao centro da figura, e o raio menor r , que corresponde ao raio da secção transversal do tubo. A sua geometria é descrita por dois ângulos, θ e ϕ , através das seguintes equações paramétricas:

$$x = (R + r \cos(\phi)) \cos(\theta)$$

$$y = r \sin(\phi)$$

$$z = (R + r \cos(\phi)) \sin(\theta)$$

3.1.1. Discretização da Superfície

A geração da malha do torus é realizada através das equações paramétricas referidas, dividindo os ângulos θ e ϕ em intervalos regulares definidos pelos parâmetros *slices* e *stacks*. O domínio de ambos os ângulos é $[0, 2\pi]$, pelo que os incrementos angulares são calculados da seguinte forma:

$$sliceStep = \frac{2\pi}{slices}$$

$$stackStep = \frac{2\pi}{stacks}$$

O primeiro parâmetro representa o incremento angular ao longo do plano horizontal do anel, enquanto o segundo divide a secção transversal do tubo em stacks intervalos iguais.

3.1.2. Definição dos Ângulos

Para a construção de cada célula da malha, consideram-se dois pares de ângulos consecutivos que definem os limites de um quadrilátero sobre a superfície do torus. Os ângulos de latitude ϕ , que definem a posição na secção transversal do tubo, são calculados a partir do índice $j \in [0, stacks - 1]$

$$\phi_1 = j \cdot stackStep \quad \phi_2 = (j + 1) \cdot stackStep$$

Os ângulos de longitude θ , que definem a rotação ao longo do anel principal, são calculados a partir do índice $i \in [0, slices - 1]$

$$\theta_1 = i \cdot sliceStep \quad \theta_2 = (i + 1) \cdot sliceStep$$

3.1.3. Triangulação da Malha

O cruzamento das divisões angulares forma uma malha composta por quadriláteros sobre a superfície do torus. No exemplo abaixo segue a representação de uma célula com vértices [ABCD]:

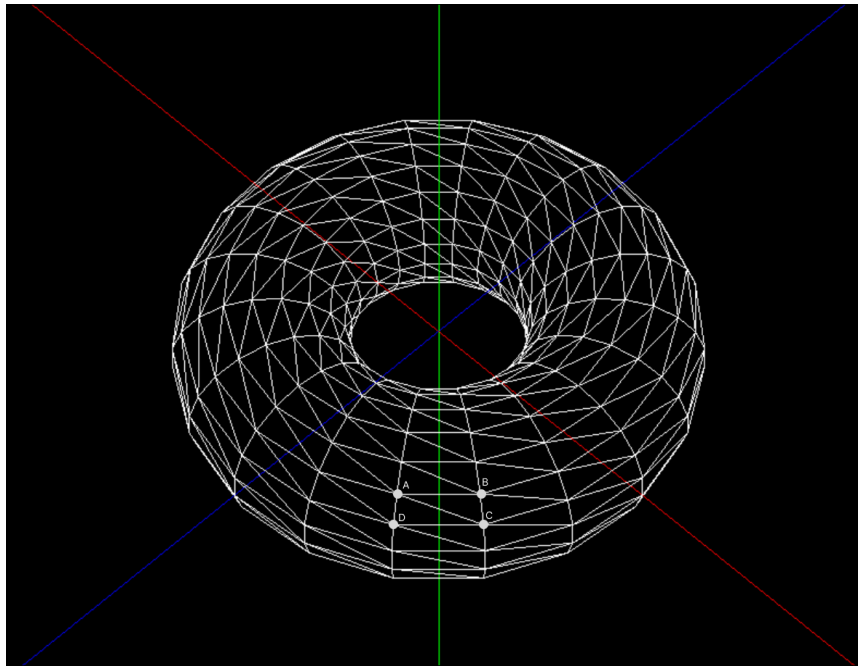


Figura 1 - Representação de um quadrilátero [ABCD] na superfície do torus

Para que os triângulos estejam voltados para fora os vértices tem que estar ordenados da seguinte forma:

$$T_1 = (A, D, C) \quad T_2 = (A, C, B)$$

Após o cálculo dos vértices parametrizados e a definição das faces triangulares para cada célula da malha, a representação geométrica contínua do torus encontra-se totalmente discretizada e concluída.

4. Sistema Solar

Nesta secção, demonstramos a construção de um cenário do Sistema Solar, utilizado para validar o suporte a hierarquias complexas no ficheiro XML e a eficiência da engine na renderização de múltiplas instâncias de modelos geométricos.

A construção do cenário do Sistema Solar foi realizada através da interpretação de um ficheiro de configuração XML, que define uma estrutura hierárquica de grupos. A engine percorre este ficheiro recursivamente, aplicando as transformações geométricas (translação, rotação e escala) ao estado da matriz atual, e carregando os modelos .3d especificados. Esta abordagem permite organizar os planetas e os seus satélites de forma relativa, herdando as transformações do seu grupo pai.

Para a geração do cinturão de asteroides, foi desenvolvido um simples programa que permitiu gerar 360 instâncias de primitivas gráficas (esferas, caixas e cones). Estas são posicionadas aleatoriamente numa coroa circular com raio entre 3.2 e 4.7 unidades, simulando a órbita entre Marte e Júpiter. O programa aplica transformações de escala reduzida e, no caso das caixas e cones, rotações aleatórias sobre o vetor $(1,1,0)$, obtendo-se uma aparência variada ao longo do cinturão.

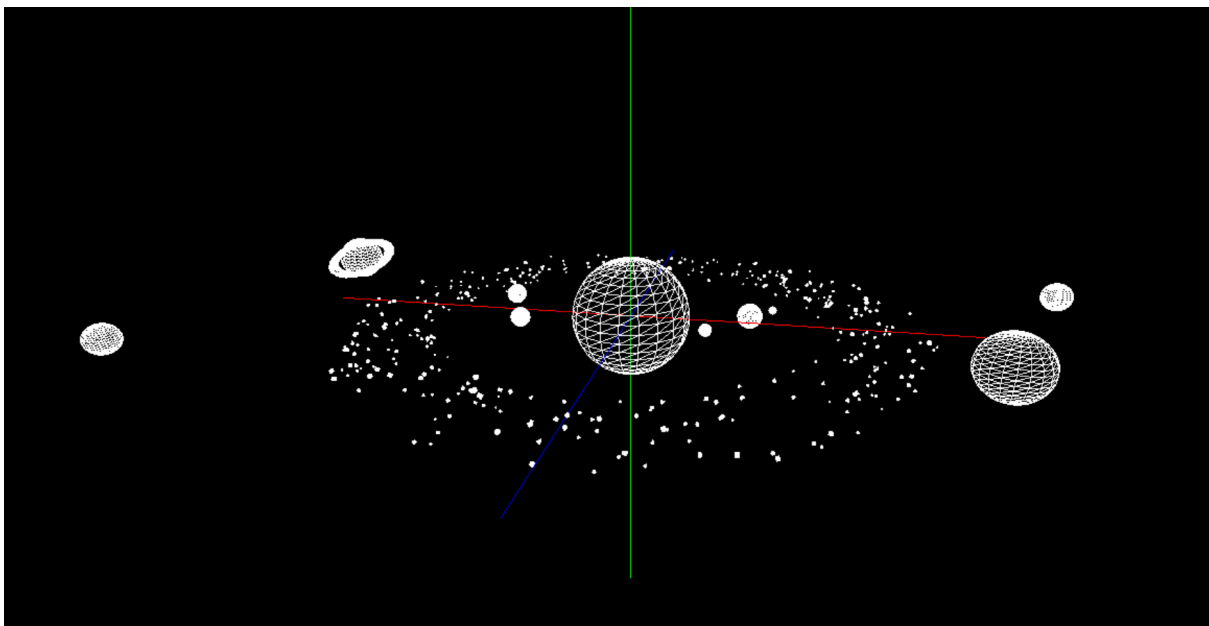


Figura 2 - Cenário do Sistema Solar

As primitivas utilizadas neste cenário, foram obtidas através dos seguintes comandos:

```
.\generator sphere 1 20 20 sphere.3d  
.\generator torus 1.2 0.2 20 20 torus.3d  
.\generator box 2 10 box.3d  
.\generator cone 1 2 10 10 cone.3d
```

Para visualizar o cenário gerado, basta executar o seguinte comando a partir da pasta *build/Release*:

```
.\engine scenes/solar.xml
```

5. Resultado dos Testes

Nesta secção, apresentam-se as imagens obtidas na renderização dos ficheiros de teste fornecidos. Os resultados demonstram que a *engine* processa corretamente a hierarquia do ficheiro XML e aplica com sucesso as transformações geométricas e a gestão do estado das matrizes de transformação.

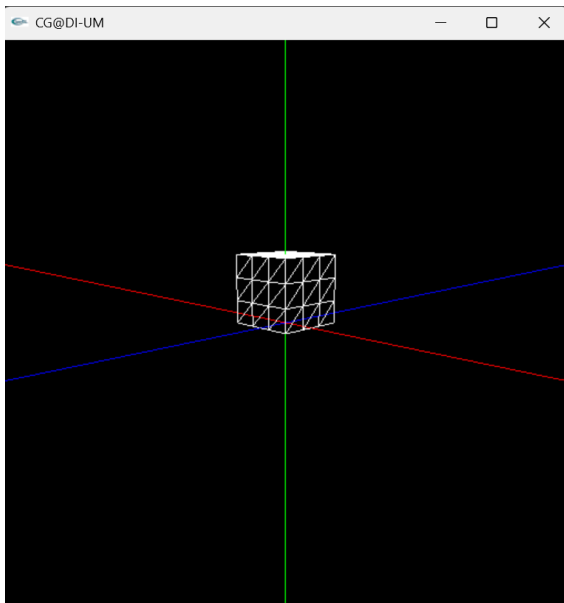


Figura 3 - Teste 1

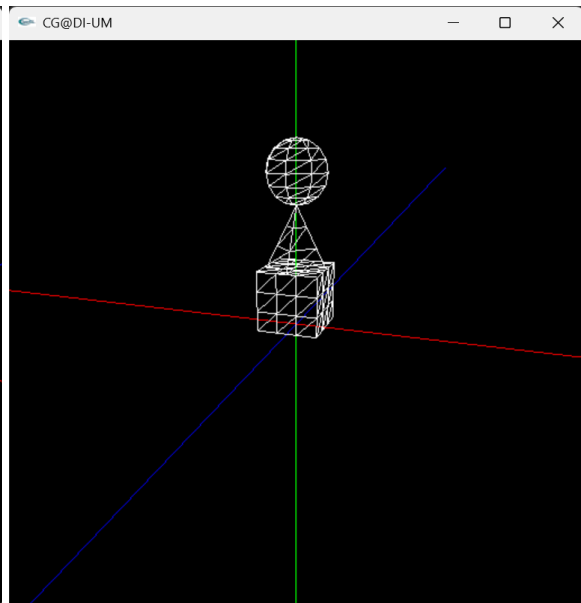


Figura 4 - Teste 2

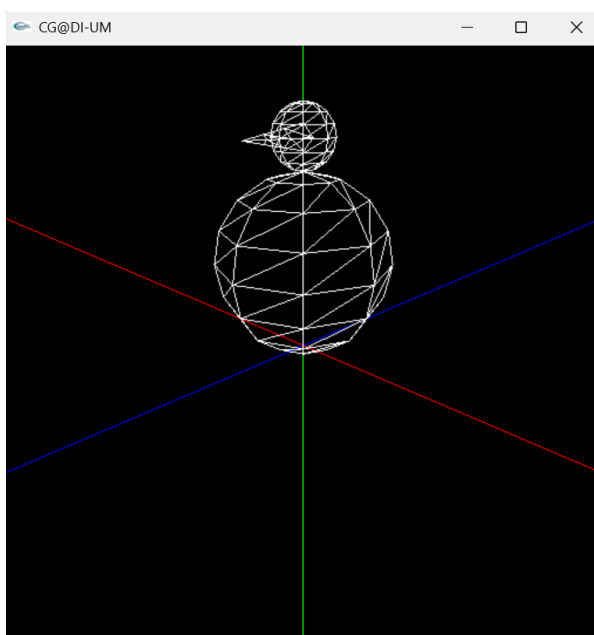


Figura 5 - Teste 3

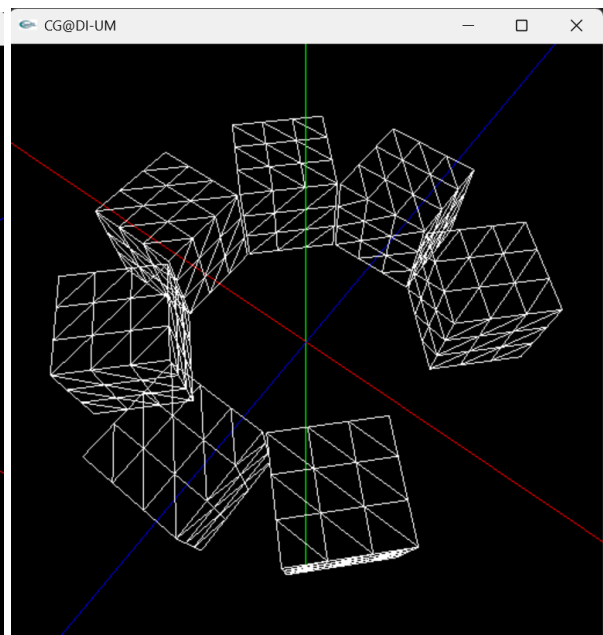


Figura 6 - Teste 4

6. Conclusão

A segunda fase do projeto de Computação Gráfica permitiu a manipulação de cenas tridimensionais complexas. O objetivo desta fase, que consistia na evolução da *engine* para suportar transformações geométricas hierárquicas, foi concluído com sucesso, através da implementação de uma estrutura de dados baseada em grupos e relações de parentesco.

A arquitetura hierárquica foi essencial para a gestão da cena, permitindo que objetos herdem as transformações dos nodos pai. Esta abordagem foi validada com a criação do cenário do Sistema Solar, onde a utilização de ficheiros XML permitiu descrever com precisão as órbitas e rotações relativas de planetas e satélites.

Por meio das estruturas *Group* e *Transformation* a *engine* foi capaz de processar sequências ordenadas de translações, rotações e escalas. O trabalho realizado nesta fase não só permitiu obter resultados visuais mais dinâmicos e articulados, como também estabelecer uma base sólida para a integração futura de técnicas de modelação e renderização mais avançadas nas fases seguintes da unidade curricular.